

VIP Cheatsheet: Transformers & Large Language Models

Afshine AMIDI and Shervine AMIDI

September 13, 2026

This VIP cheatsheet gives an overview of what is in the "Super Study Guide: Transformers & Large Language Models" book, which contains ~600 illustrations over 250 pages and goes into the following concepts in depth. You can find more details at <https://superstudy.guide>.

1 Foundations

1.1 Tokens

Definition – A *token* is an indivisible unit of text, such as a word, subword or character, and is part of a predefined vocabulary.

Remark: The unknown token [UNK] represents unknown pieces of text while the padding token [PAD] is used to fill empty positions to ensure consistent input sequence lengths.

Tokenizer – A *tokenizer* T divides text into tokens of an arbitrary level of granularity.

this teddy bear is reaaaaally cute $\rightarrow$ T $\rightarrow$ [this] [teddy bear] [is] [UNK] [cute] [PAD] ... [PAD]

Here are the main types of tokenizers:

Type	Pros	Cons	Illustration
Word	- Easy to interpret - Short sequence	- Large vocabulary size - Word variations not handled	[teddy] [bear]
Subword	- Word roots leveraged - Intuitive embeddings	- Increased sequence length - Tokenization more complex	[ted] [##dy] [bear]
Character	No out-of-vocabulary concerns	Much longer sequence length	[t] [e] [d] [d] [y]
Byte	Small vocabulary size	Patterns hard to interpret because too low-level	[b] [e] [a] [r]

Remark: Byte-Pair Encoding (BPE) and Unigram are commonly-used subword-level tokenizers.

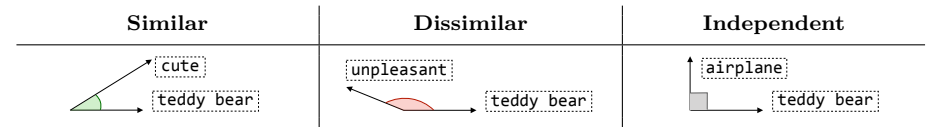
1.2 Embeddings

Definition – An *embedding* is a numerical representation of an element (e.g. token, sentence) and is characterized by a vector $x \in \mathbb{R}^n$.

Similarity – The *cosine similarity* between two tokens t_1, t_2 is quantified by:

$$\text{similarity}(t_1, t_2) = \frac{t_1 \cdot t_2}{\|t_1\| \|t_2\|} = \cos(\theta) \in [-1, 1]$$

The angle θ characterizes the similarity between the two tokens:

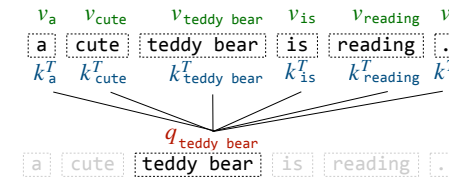


Remark: Approximate Nearest Neighbors (ANN) and Locality Sensitive Hashing (LSH) are methods that approximate the similarity operation efficiently over large databases.

2 Transformers

2.1 Attention

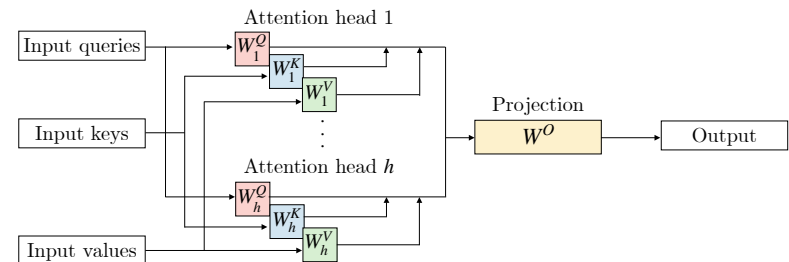
Formula – Given a *query* q , we want to know which *key* k the query should pay "attention" to with respect to the associated *value* v .



Attention can be efficiently computed using matrices Q, K, V that contain queries q , keys k and values v respectively, along with the dimension d_k of keys:

$$\text{attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

MHA – A *Multi-Head Attention* (MHA) layer performs attention computations across multiple heads, then projects the result in the output space.

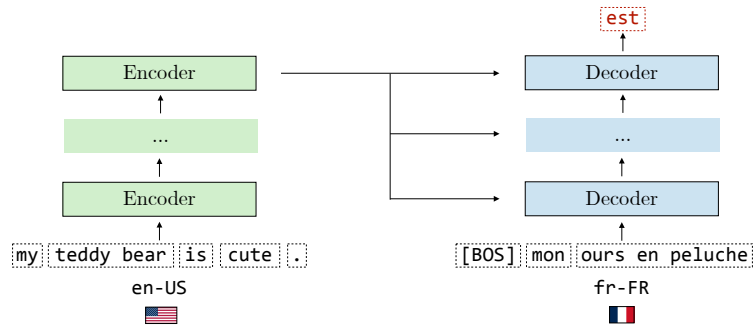


It is composed of h attention heads as well as matrices W^Q, W^K, W^V that project the input to obtain queries Q , keys K and values V . The projection is done using matrix W^O .

Remark: Grouped-Query Attention (GQA) and Multi-Query Attention (MQA) are variations of MHA that reduce computational overhead by sharing keys and values across attention heads.

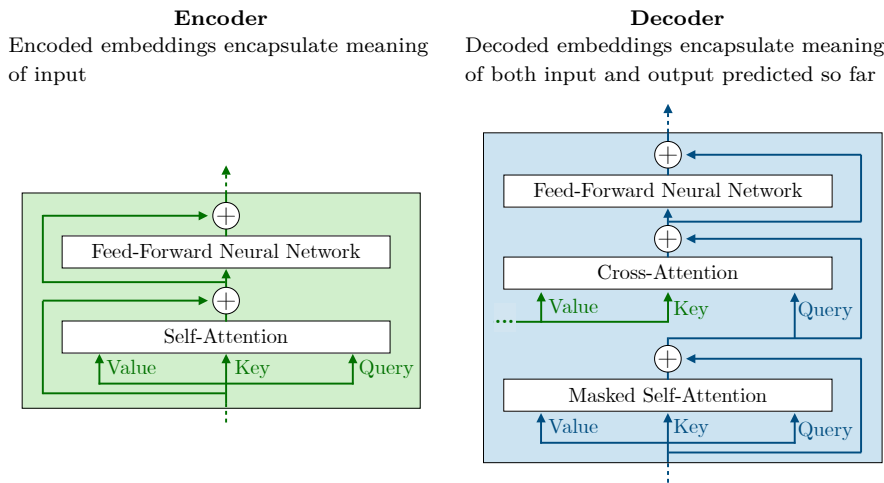
2.2 Architecture

Overview – The *Transformer* is a landmark model relying on the self-attention mechanism and is composed of encoders and decoders. Encoders compute meaningful embeddings of the input that are then used by decoders to predict the next token in the sequence.



Remark: Although the Transformer was initially proposed as a model for translation tasks, it is now widely used across many other applications.

□ **Components** – The *encoder* and *decoder* are two fundamental components of the Transformer and have different roles:

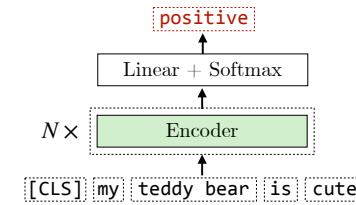


□ **Position embeddings** – *Position embeddings* inform where the token is in the sequence. They can either be arbitrarily defined or learned from the data.

Remark: Rotary Position Embeddings (RoPE) are a popular and efficient variation that rotate query and key vectors to incorporate relative position information.

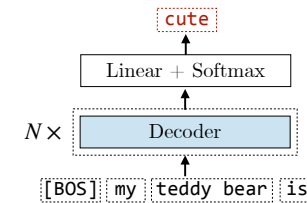
2.3 Variants

□ **Encoder-only** – *Bidirectional Encoder Representations from Transformers* (BERT) is a Transformer-based model composed of a stack of encoders that takes some text as input, and outputs meaningful embeddings, which can be later used in downstream classification tasks.



A [CLS] token is added at the beginning of the sequence to capture the meaning of the sentence. Its encoded embedding is often used in downstream tasks, such as sentiment extraction.

□ **Decoder-only** – *Generative Pre-trained Transformer* (GPT) is an autoregressive Transformer-based model that is composed of a stack of decoders. Contrary to BERT and its derivatives, GPT treats all problems as text-to-text problems.



Most of the current state-of-the-art LLMs rely on a decoder-only architecture, such as the GPT series, DeepSeek, Mistral, Gemma, etc.

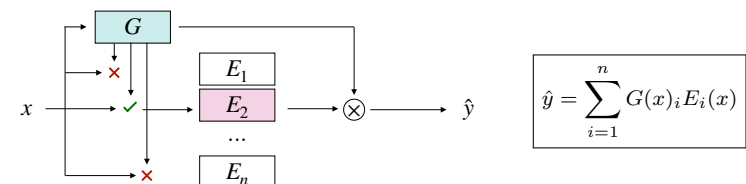
Remark: Encoder-decoder models, like T5, are also autoregressive and share many characteristics with decoder-only models.

3 Large language models

3.1 Overview

□ **Definition** – A *Large Language Model* (LLM) is a Transformer-based model with strong NLP capabilities. It is "large" in the sense that it typically contains billions of parameters and is trained on a large amount of data.

□ **Mixture of experts** – A *Mixture of Experts* (MoE) is a model that activates only a portion of its neurons at inference time. It is based on a gate G and experts $E_1, \dots, E_n$.



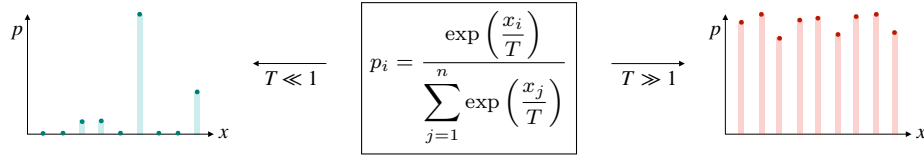
MoE-based LLMs use this gating mechanism in their FFNNs to route each token to a subset of experts.

3.2 Prompting

□ **Context length** – The *context length* of a model is the maximum number of tokens that can fit into the input. It typically ranges from tens of thousands to millions of tokens.

Remark: LLM response quality tends to degrade as the context window gets filled up, which is a phenomenon known as "context rot".

□ **Decoding sampling** – Token predictions are sampled from the predicted probability distribution p_i , which is controlled by the hyperparameter temperature T .



Remark: High temperatures lead to more creative outputs whereas low temperatures lead to more deterministic ones.

4 LLM training

4.1 Pretraining

□ **Cross-entropy loss** – *Cross-Entropy* (CE) is a loss function that penalizes the model when it assigns low probability p_i to the correct next token i :

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^V y_i \log p_i$$

□ **Scaling laws** – *Scaling laws* are empirical laws that describe predictable power-law improvements in model performance as model size, training data and compute are increased.

4.2 Finetuning

□ **SFT** – *Supervised FineTuning* (SFT) is a post-training approach that aligns the behavior of the model with an end task. It relies on high-quality input-output pairs aligned with the task.

Remark: If the SFT data is about instructions, then this step is called "instruction tuning".

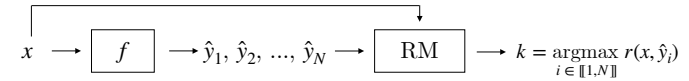
□ **PEFT** – *Parameter-Efficient FineTuning* (PEFT) is a category of methods used to run SFT efficiently. In particular, *Low-Rank Adaptation* (LoRA) approximates the learnable weights W by fixing W_0 and learning low-rank matrices A, B instead:

$$\begin{matrix} d & \xrightarrow{k} \\ \text{ } & W \end{matrix} \approx \begin{matrix} d & \xrightarrow{k} \\ \text{ } & W_0 \end{matrix} + \begin{matrix} d & \xrightarrow{r} \\ \text{ } & B \end{matrix} \times \begin{matrix} r & \xrightarrow{k} \\ \text{ } & A \end{matrix}$$

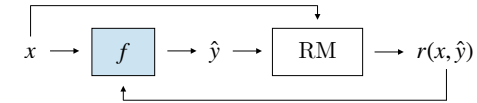
Remark: Other PEFT techniques include prefix tuning and adapter layer insertion.

4.3 Preference tuning

□ **Reward model** – A *Reward Model* (RM) is a model that predicts how well an output $\hat{y}$ aligns with desired behavior given the input x . *Best-of-N* (BoN) sampling, also called *rejection sampling*, is a method that uses a reward model to select the best response among N generations.



□ **Reinforcement learning** – *Reinforcement Learning* (RL) is an approach that leverages RM and updates the model f based on rewards for its generated outputs. If RM is based on human preferences, this process is called *Reinforcement Learning from Human Feedback* (RLHF).



Remark: There are also supervised approaches, like Direct Preference Optimization (DPO), that combine RM and RL into one supervised step.

4.4 Reasoning

□ **Chain-of-thought** – *Chain-of-Thought* (CoT) is a reasoning process in which the model breaks down a complex problem into a series of intermediate steps. This helps the model to generate the correct final response.

□ **Reasoning model** – A *reasoning model* is a model that relies on CoT-based reasoning traces to solve more complex tasks in math, coding and logic. Two types of scaling methods are used to enhance reasoning capabilities:

	Description	Illustration
Train-time scaling	Run RL for longer to let the model learn how to produce CoT-style reasoning traces before giving an answer	
Test-time scaling	Let the model think longer before providing an answer with budget forcing keywords such as "Wait"	

□ **OPD** – *On-Policy Distillation* (OPD) is a post-training approach that relies on trajectories sampled from the model and that uses a stronger teacher to provide token-level supervision on those trajectories.

Remark: Unlike sparse sequence-level rewards, OPD provides a dense token-level learning signal from the teacher throughout the generated trajectory.

5 Reinforcement learning with LLMs

5.1 Fundamentals

□ **Setup** – In applications of RL with LLMs:

- The *agent* is the language model, whose *policy* defines the probability distribution over next-token actions.
- The *state* is the prompt plus generated tokens so far.
- The *action* is the next sampled token.
- The *reward* scores the generated response according to the desired behavior or task outcome.

□ **Policy gradients** – *Policy gradient* is a family of RL algorithms that updates a policy to favor actions with higher sampled returns R . Since R can vary significantly across trajectories, we can use a baseline $b(s)$ to quantify actions that perform better than expected $R - b(s)$. This helps reduce the variance in the update.

Remark: REINFORCE is an example of a policy-gradient algorithm.

5.2 PPO

□ **Value function** – A *value function* estimates the expected future reward from a given state or state-action pair under the current policy.

□ **Algorithm** – *Proximal Policy Optimization* (PPO) is a popular RL algorithm that incentivizes higher rewards while keeping the model close to the base model to prevent reward hacking. By defining the policy probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, we have:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

5.3 GRPO

□ **Verifiable rewards** – *Verifiable rewards* are automatically computed rewards based on objectively checkable outcomes, such as exact answers, code execution, or formal proofs.

□ **Algorithm** – *Group Relative Policy Optimization* (GRPO) estimates policy advantages $\hat{A}_i$ by comparing rewards R_i across a group of G sampled responses, avoiding the need for a separate value model. By defining $\hat{A}_i = \frac{R_i - \text{mean}(R_1, \dots, R_G)}{\text{std}(R_1, \dots, R_G)}$, we have:

$$\mathcal{L}_{\text{GRPO}} = \frac{1}{G} \sum_{i=1}^G \mathbb{E}_t \left[\min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \right]$$

Remark: Extensions exist to address limitations of GRPO. For example, Dr. GRPO corrects length-related bias, while DAPO improves training stability and exploration through modified clipping and dynamic sampling.

6 LLM systems

6.1 Kernels

□ **Flash attention** – *Flash attention* is an exact method that optimizes attention computations by cleverly leveraging GPU hardware, using the fast *Static Random-Access Memory* (SRAM) for matrix operations before writing results to the slower *High Bandwidth Memory* (HBM).

Remark: In practice, this reduces memory usage and speeds up computations.

□ **Paged attention** – *Paged attention* manages the KV cache in fixed-size memory blocks, reducing memory fragmentation and enabling efficient memory sharing across sequences.

6.2 Distributed training

□ **Parallelism** – *Parallelism* distributes LLM training across multiple devices by partitioning data, model computation, or model components.

□ **Types** – The main forms of parallelism are:

- *Data parallelism*: Replicate the model and split training batches across devices.
- *Tensor parallelism*: Split computations within individual layers across devices.
- *Pipeline parallelism*: Split model layers into stages placed on different devices.
- *Expert parallelism*: Distribute MoE experts across devices and route tokens accordingly.

Remark: Large-scale LLM training typically combines several forms of parallelism.

6.3 Inference optimizations

□ **KV caching** – *Key-Value caching* stores previously computed key and value vectors during decoding, avoiding redundant attention computations when generating each new token.

Remark: Prompt caching reuses KV cache entries for repeated prompt prefixes.

□ **Prefill vs decode** – LLM inference consists of two phases: *prefill*, which processes all prompt tokens in parallel to initialize the KV cache, and *decode*, which generates new tokens autoregressively one at a time.

□ **Speculative decoding** – *Speculative decoding* is a method that accelerates generation by using a smaller draft model to propose multiple tokens that are then verified in parallel by the target model.

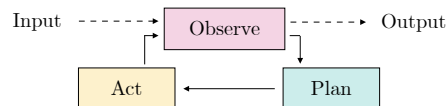
Remark: This method is speeding up the decoding process because proposing tokens with a small model is cheap, while the target model can verify several proposed tokens in a single forward pass instead of generating them one by one.

7 AI Agents

7.1 Overview

□ **Definition** – An *AI agent* is a system that autonomously pursues goals and completes tasks on a user's behalf. It may use different chains of LLM calls to do so.

□ **ReAct** – *Reason + Act* (ReAct) is a framework that allows for multiple chains of LLM calls to complete complex tasks:



This framework is composed of the steps below:

- *Observe*: Synthesize previous actions and explicitly state what is currently known.
- *Plan*: Detail what tasks need to be accomplished and what tools to call.
- *Act*: Perform an action via an API or look for relevant information in a knowledge base.

Remark: Evaluating an agentic system is challenging. However, this can still be done both at the component level via local inputs-outputs and at the system level via chains of calls.

□ **Harness** – The *harness* is the system around the model that manages context, tools, state and the agent loop.

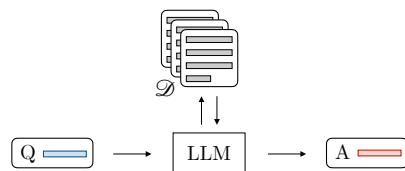
Remark: Examples include Codex, Claude Code and Pi.

7.2 Retrieval

□ **Retriever** – A *retriever* selects relevant documents from a knowledge base, typically using *sparse* retrieval (keyword-based), *dense* retrieval (embedding-based), or a hybrid of both.

□ **Reranker** – A *reranker* reorders retrieved documents using a more accurate relevance model, improving the quality of the context passed to the LLM.

□ **RAG** – *Retrieval-Augmented Generation* (RAG) is a method that allows the LLM to access relevant external knowledge to answer a given question. This is particularly useful if we want to incorporate information past the LLM pretrained knowledge cut-off date.



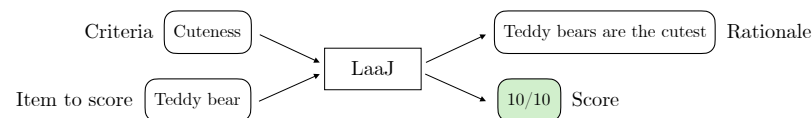
Given a knowledge base $\mathcal{D}$ and a question, a **Retriever** fetches the most relevant documents, then **Augments** the prompt with the relevant information before **Generating** the output.

Remark: RAG quality depends heavily on document chunking, retrieval quality and the number of retrieved documents included in context.

8 Evaluation

8.1 LLM-as-a-Judge

□ **LLM-as-a-Judge** – *LLM-as-a-Judge* (LaaJ) is a method that uses an LLM to score given outputs according to some provided criteria. Notably, it is also able to generate a rationale for its score, which helps with interpretability.



Contrary to pre-LLM era metrics such as *Recall-Oriented Understudy for Gisting Evaluation* (ROUGE), LaaJ does not need any reference text, which makes it convenient to evaluate on any kind of task. In particular, LaaJ shows strong correlation with human ratings when it relies on a big powerful model (e.g. GPT-6), as it requires reasoning capabilities to perform well.

Remark: LaaJ is useful to perform quick rounds of evaluations but it is important to monitor the alignment between LaaJ outputs and human evaluations to make sure there is no divergence.

□ **Common biases** – LaaJ models can exhibit the following biases:

	Position bias	Verbosity bias	Self-enhancement bias
Problem	Favors first position in pairwise comparisons	Favors more verbose content	Favors outputs generated by themselves
Solution	Average metric on randomized positions	Add a penalty on the output length	Use a judge built from a different base model

Remark: The list of biases above is not exhaustive.

8.2 Agent evaluation

□ **Component** – *Component*-level evaluation measures the quality of individual agent modules, such as retrieval, tool selection or action execution.

Remark: Key metrics include accuracy, precision, recall, tool-call success rate and latency.

□ **Trajectory** – *Trajectory*-level evaluation assesses the full sequence of observations, decisions and actions taken by an agent.

Remark: Key metrics include task success rate, number of steps/tool calls, error rate, latency and cost.

9 Diffusion LLMs

9.1 Paradigm

□ **ARM** – An *AutoRegressive Model* (ARM) generates a sequence one token at a time, conditioning each new token on all previously generated tokens.

□ **Diffusion LLM** – A *diffusion LLM* (dLLM) generates text through an iterative denoising process on the whole sequence. This contrasts with ARMs which generate tokens sequentially from left to right. The two main types of dLLMs are:

- *Continuous diffusion*: Denoises continuous token embeddings, then maps resulting embeddings to discrete tokens.
- *Discrete diffusion*: Denoises tokens directly in the discrete space.

Remark: Discrete diffusion is currently the most common formulation for diffusion LLMs, particularly through masked diffusion, where tokens are progressively masked and reconstructed.

9.2 Training

□ **Objective** – The model is trained to reconstruct the original tokens from a corrupted sequence at a randomly sampled noise level.

□ **Forward process** – The *forward process* progressively corrupts clean text, e.g. by replacing an increasing fraction of tokens with [MASK].

9.3 Inference

□ **Strategy** – At inference time, a diffusion LLM starts from a corrupted or masked sequence and repeatedly predicts tokens until a complete response is obtained.

Remark: Diffusion LLMs naturally support arbitrary-order generation and infilling, since generation is not constrained to proceed strictly from left to right.

□ **Decoding schedule** – A *decoding schedule* determines which predicted tokens are accepted or remasked at each denoising step, trading off generation speed and quality.